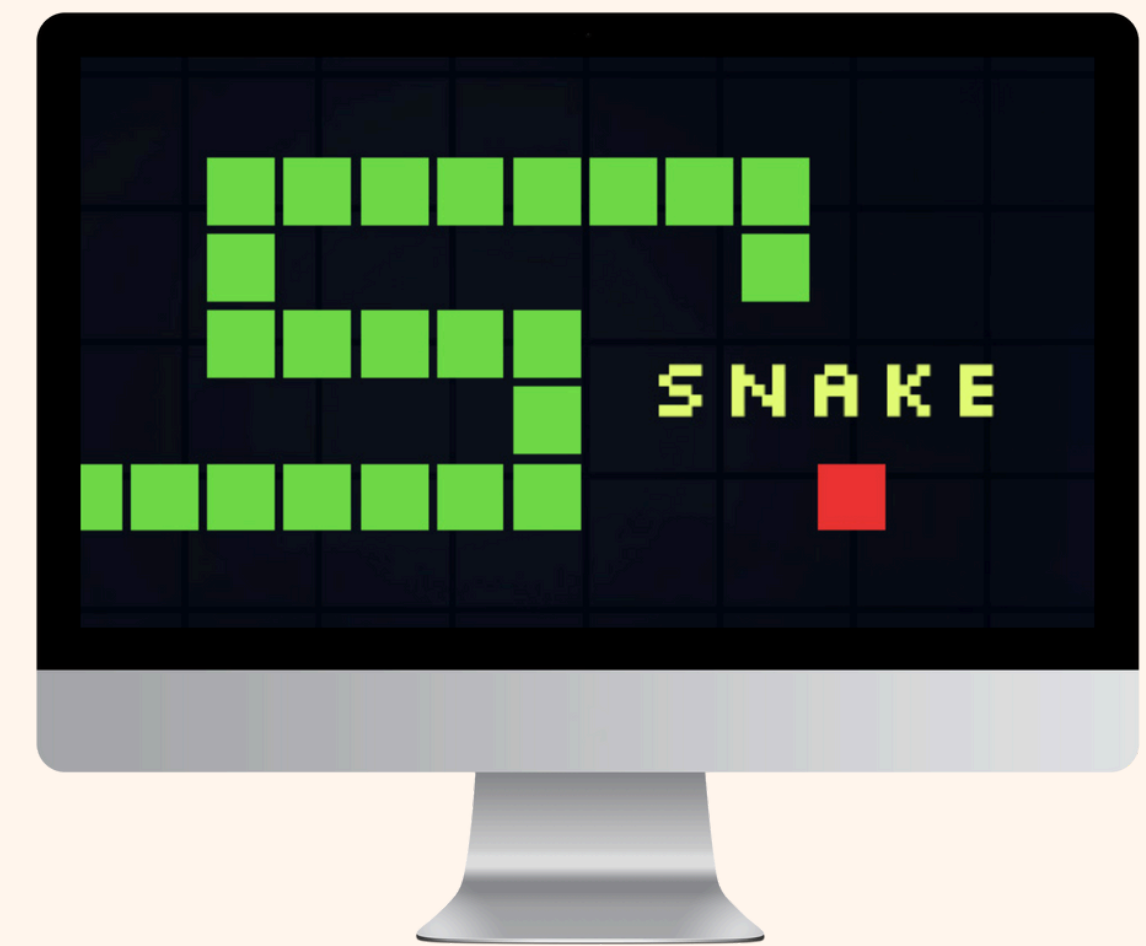


REINFORCEMENT LEARNING

SNAKE CU OBSTACOLE

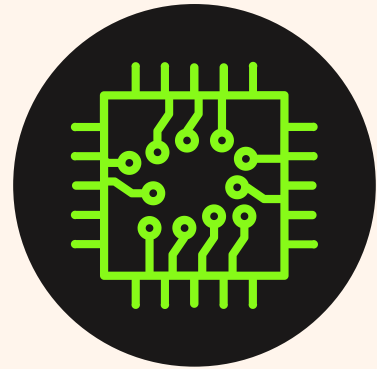
Echipa:

Damian Alexandru
Horneț Alex Andrei
Rizea Mihai Marius
Zamfir Cezara Maria



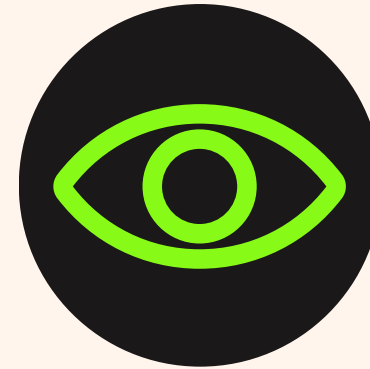
Proiect RL 2025–2026

ALEGEREA TEMEI: De ce Snake?



Problemă clasică de RL

Exemplu perfect de Trial&Error: agentul trebuie să descopere singur o strategie câștigătoare, fără să fie programat explicit.



Destul de complex, dar ușor de înțeles

Deși spațiul de stări este mare și agentul trebuie să planifice pe termen lung, putem vizualiza și interpreta ușor ce învață și unde greșește.



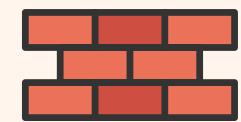
Excelent pentru compararea algoritmilor

Mediul simplu și consistent ne permite să testăm diferiți agenți în aceleași condiții și să vedem diferențele în performanță.

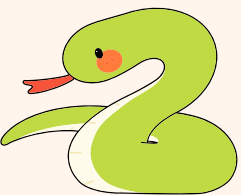
FORMULAREA PROBLEMEI

Am ales să modelăm jocul ca un **Proces Decizional Markov (MDP)**.

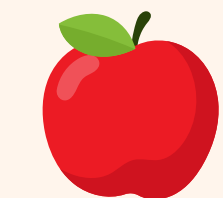
STAREA (11 valori binare)



Pericol în față/la stânga/la dreapta?

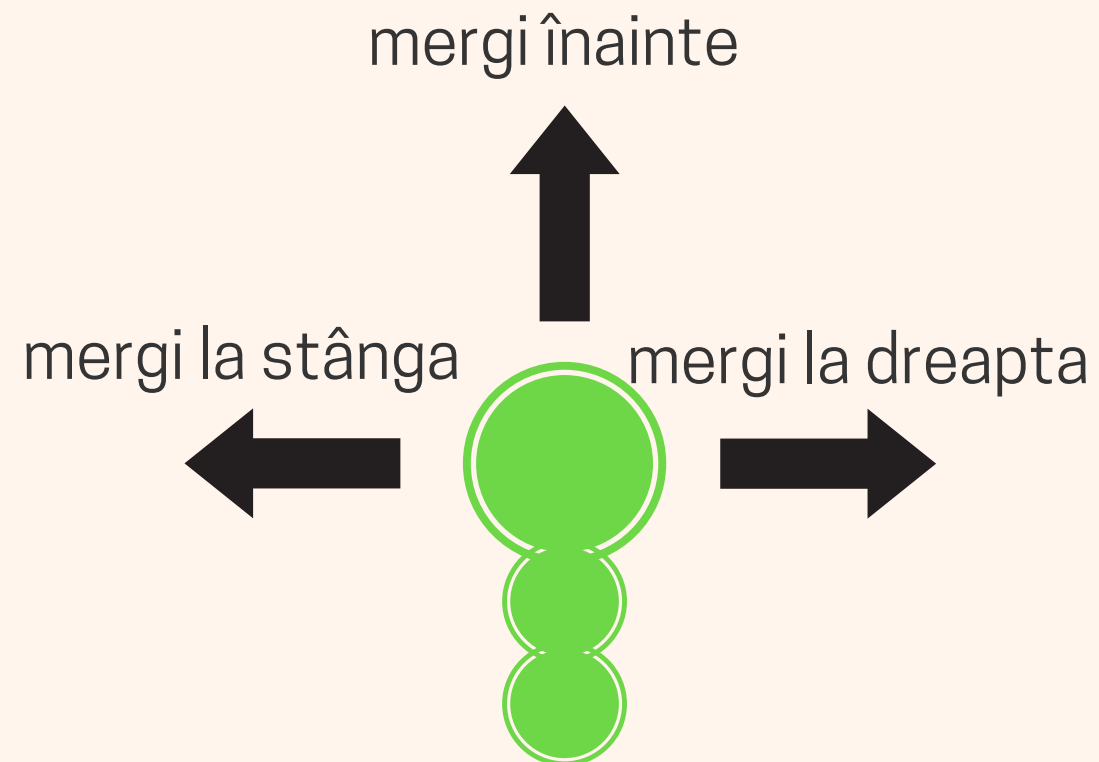


În ce direcție mă mișc?
(sus/jos/stânga/dreapta)



Unde e mâncarea?
(sus/jos/stânga/dreapta față de mine)

ACȚIUNILE



RECOMPENSELE

+10
pentru hrană

-10
pentru obstacol (zid/corp)

+0.1/-0.1
pentru
apropierea/îndepărătarea
de hrană

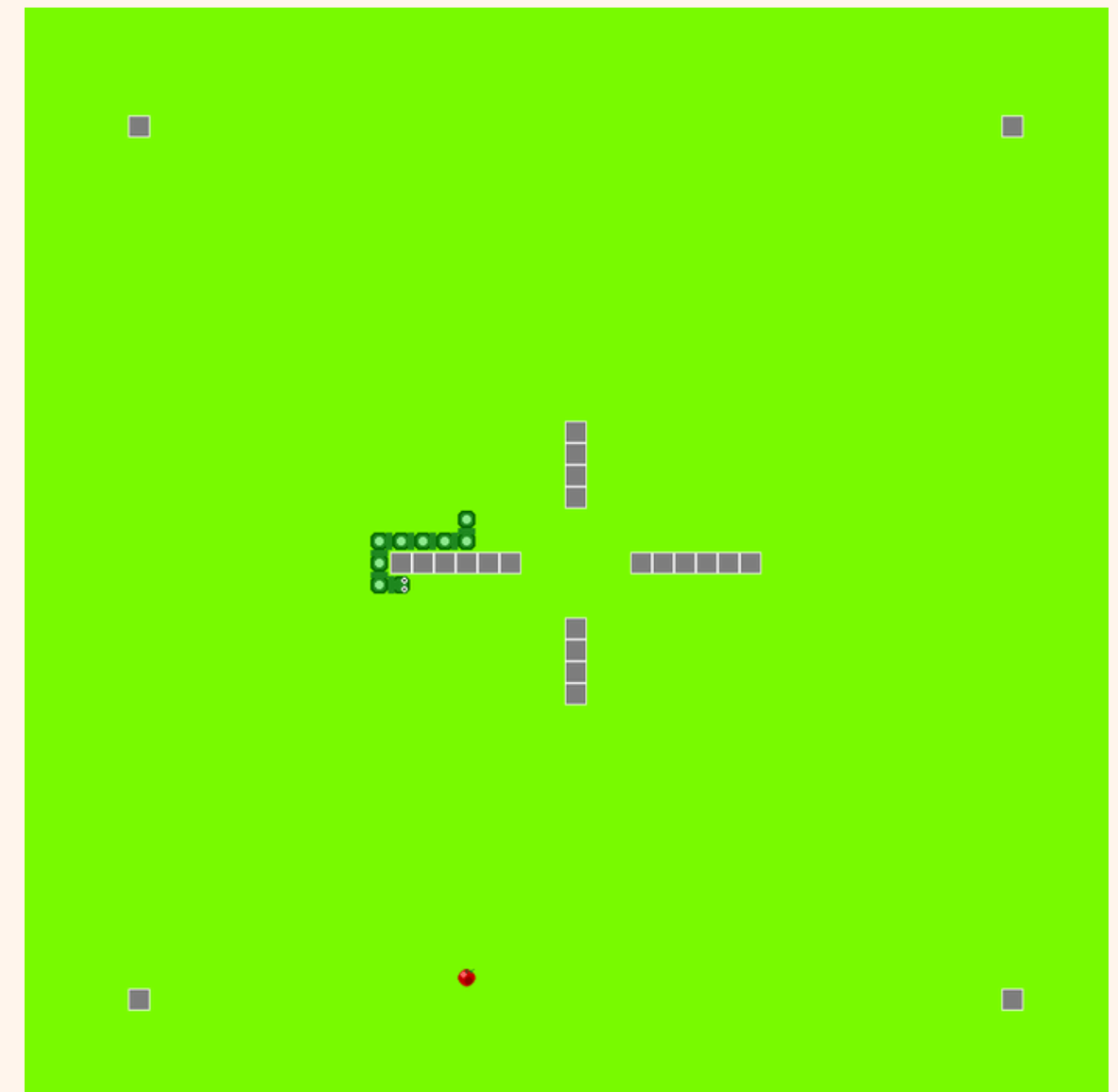
ENVIRONMENT

Am creat un environemt personalizat în PyGame, cu o modificare față de Snake-ul clasic: am introdus **obstacole fixe** pentru a asigura reproductibilitatea experimentelor.

Curriculum Learning (scalarea mediului de testare)

Harta crește de la 200px (Small) pentru învățarea rapidă a regulilor, până la 1000px (XXL).

Facem asta pentru a valida că agentul generalizează strategia și nu doar memorează trasee scurte.



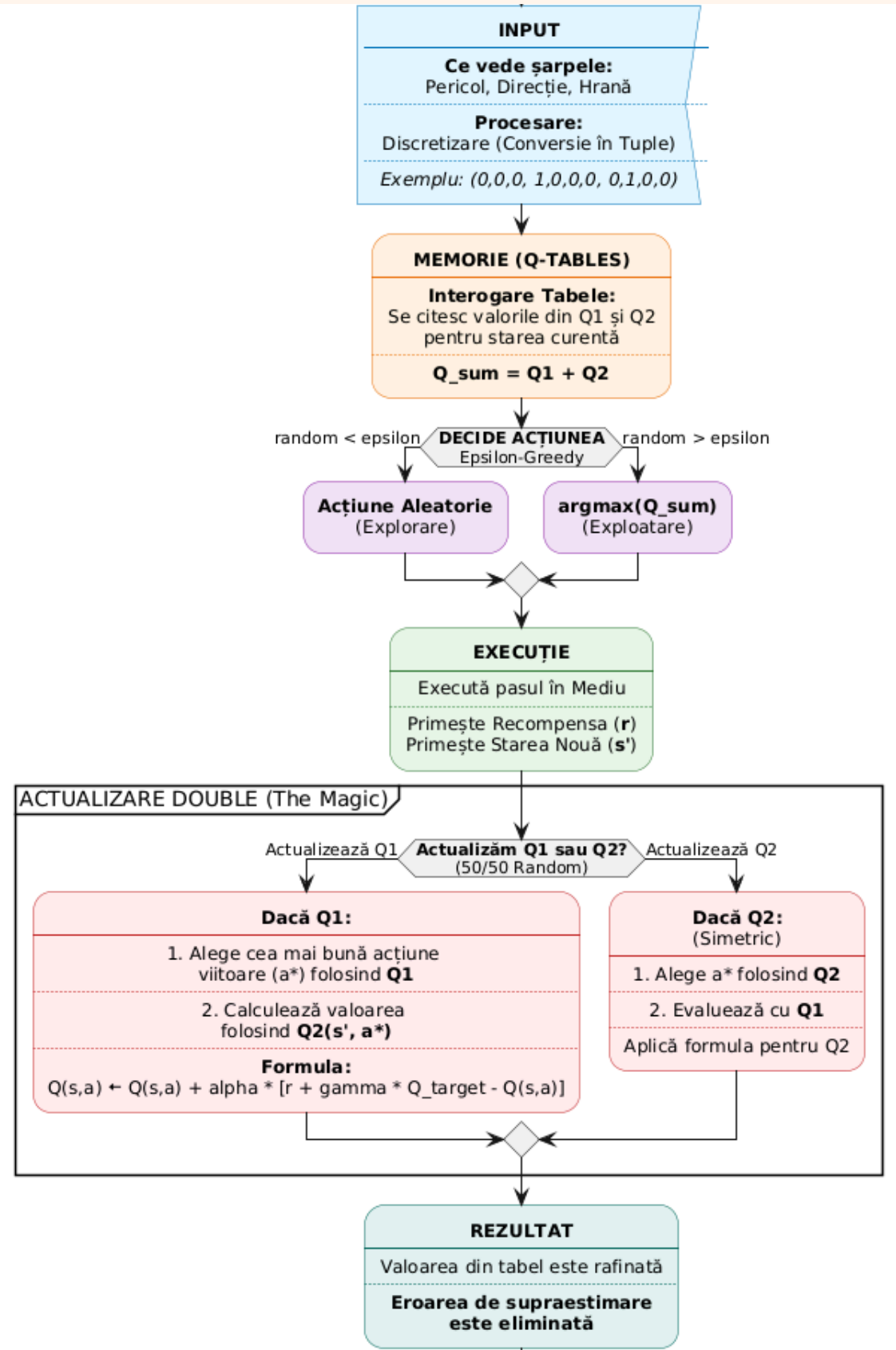


AGENT 1: DOUBLE Q-LEARNING

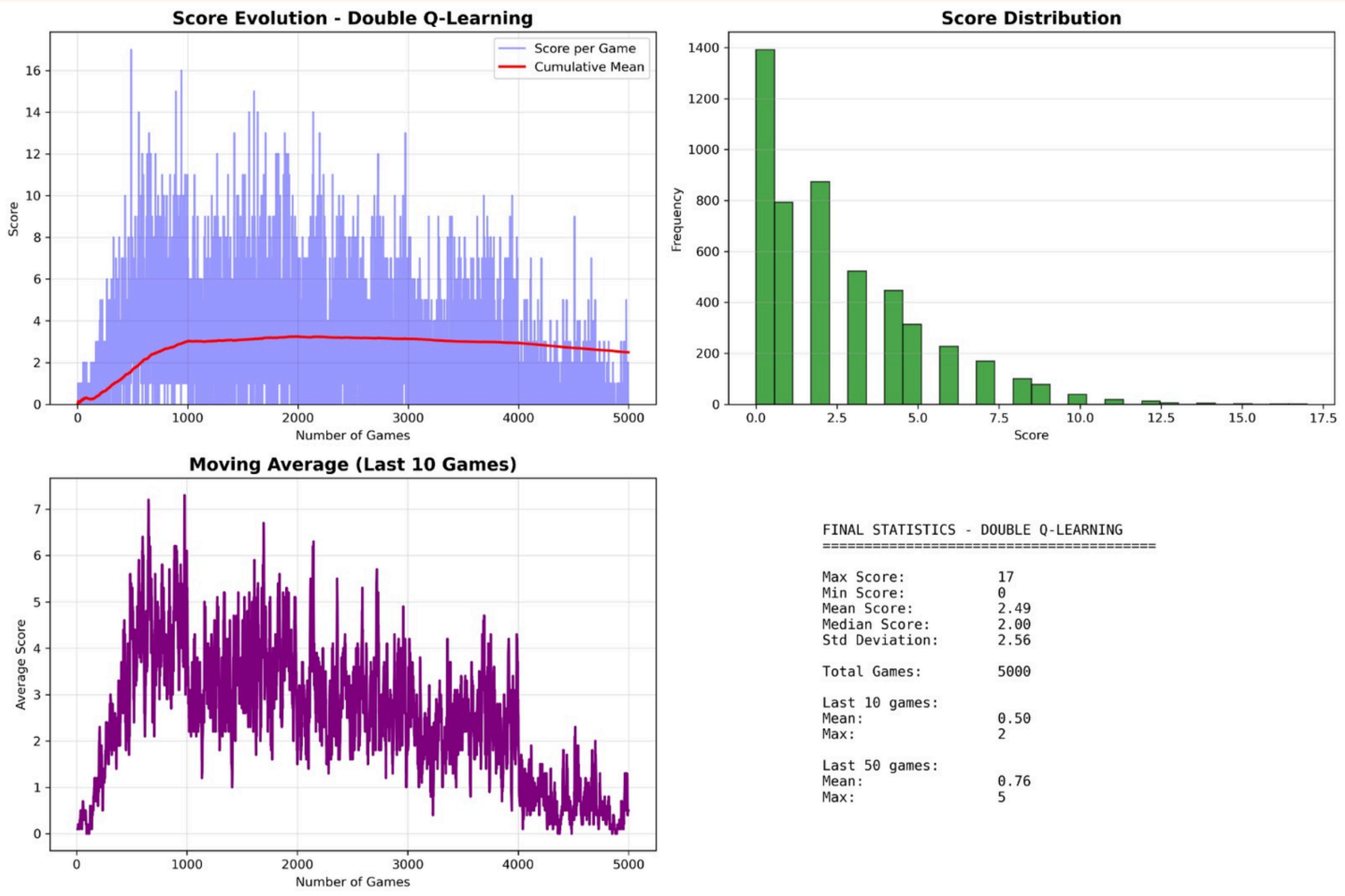
Double Q-Learning este o metodă tabulară (fără rețele neurale) care rezolvă problema supraestimării valorilor ("Maximization Bias") specifică Q-Learning-ului clasic.

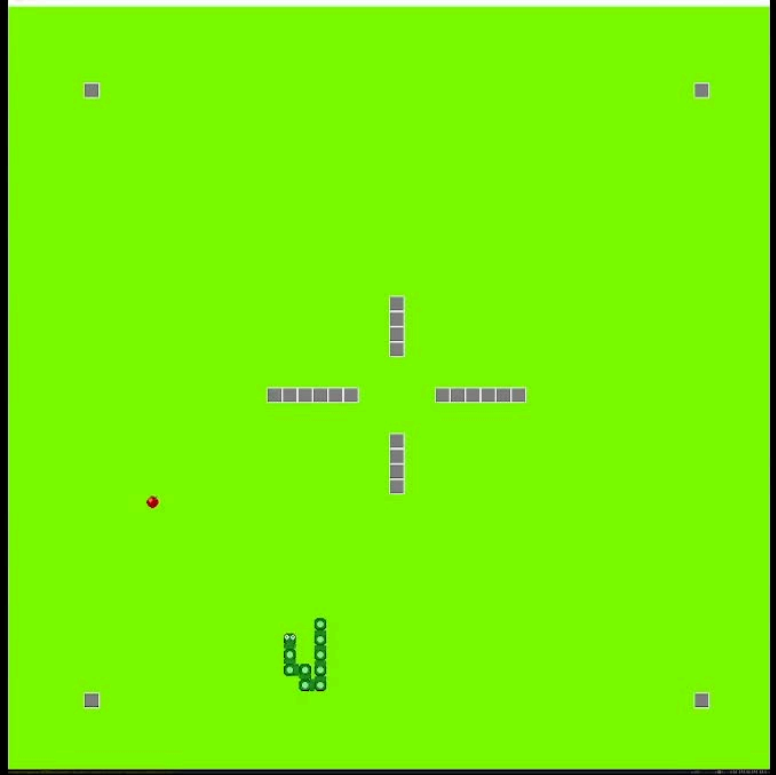
Folosește două tabele separate (Q1 și Q2) pentru a decupla selecția acțiunii de evaluarea acesteia: Q1 alege cea mai bună mutare, iar Q2 îi estimează valoarea.

Hiperparametru	Valoare	Rol
Gamma (γ)	0.95	Balanță între recompensa imediată și viitoare
Learning Rate (α)	Dinamic (0.5 $\rightarrow$ 0.1)	Viteza de actualizare a valorilor Q (scade în timp)
Epsilon Start (ϵ)	0.8 (80%)	Explorare agresivă la începutul antrenamentului
Epsilon Decay	0.995	Tranziție lentă spre exploatare (strategie stabilă)
Replay Memory	100,000	Stochează experiențele pentru Prioritized Replay
Batch Size	1000	Numărul de situații re-învățate la fiecare pas



AGENT 1: DOUBLE Q-LEARNING





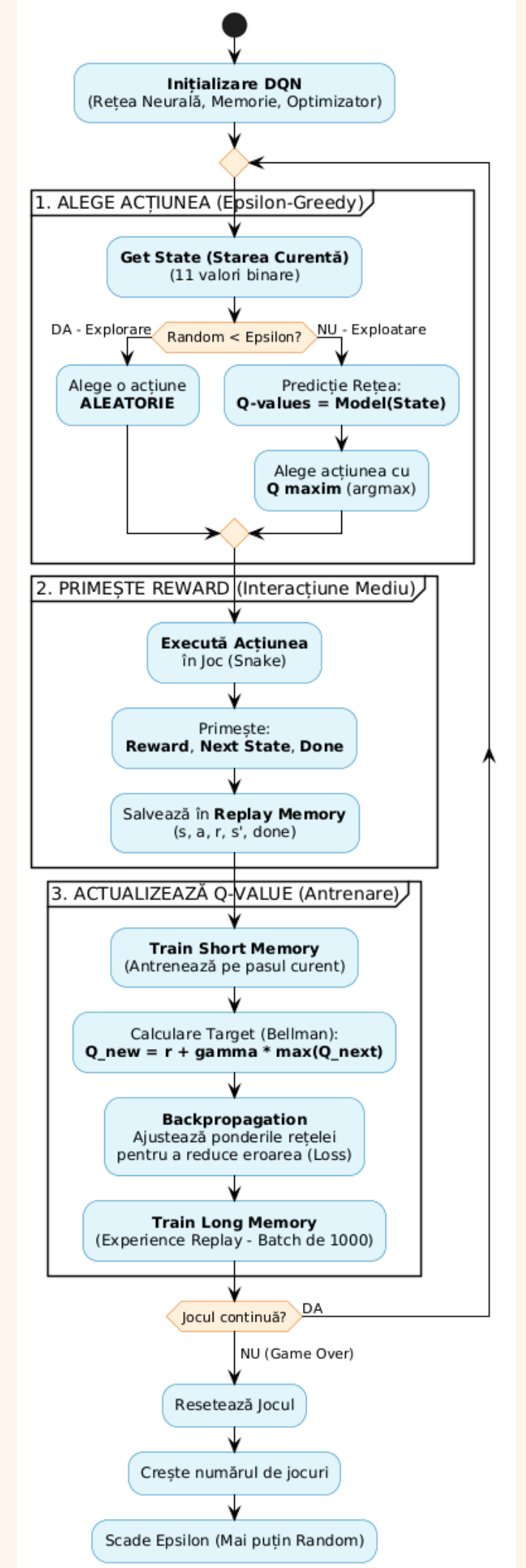
AGENT 2: DQN

ALEGE ACȚIUNE → PRIMEȘTE REWARD →
ACTUALIZEAZĂ Q-VALUE → REPETĂ PÂNĂ ÎNVAȚĂ POLITICA OPTIMĂ

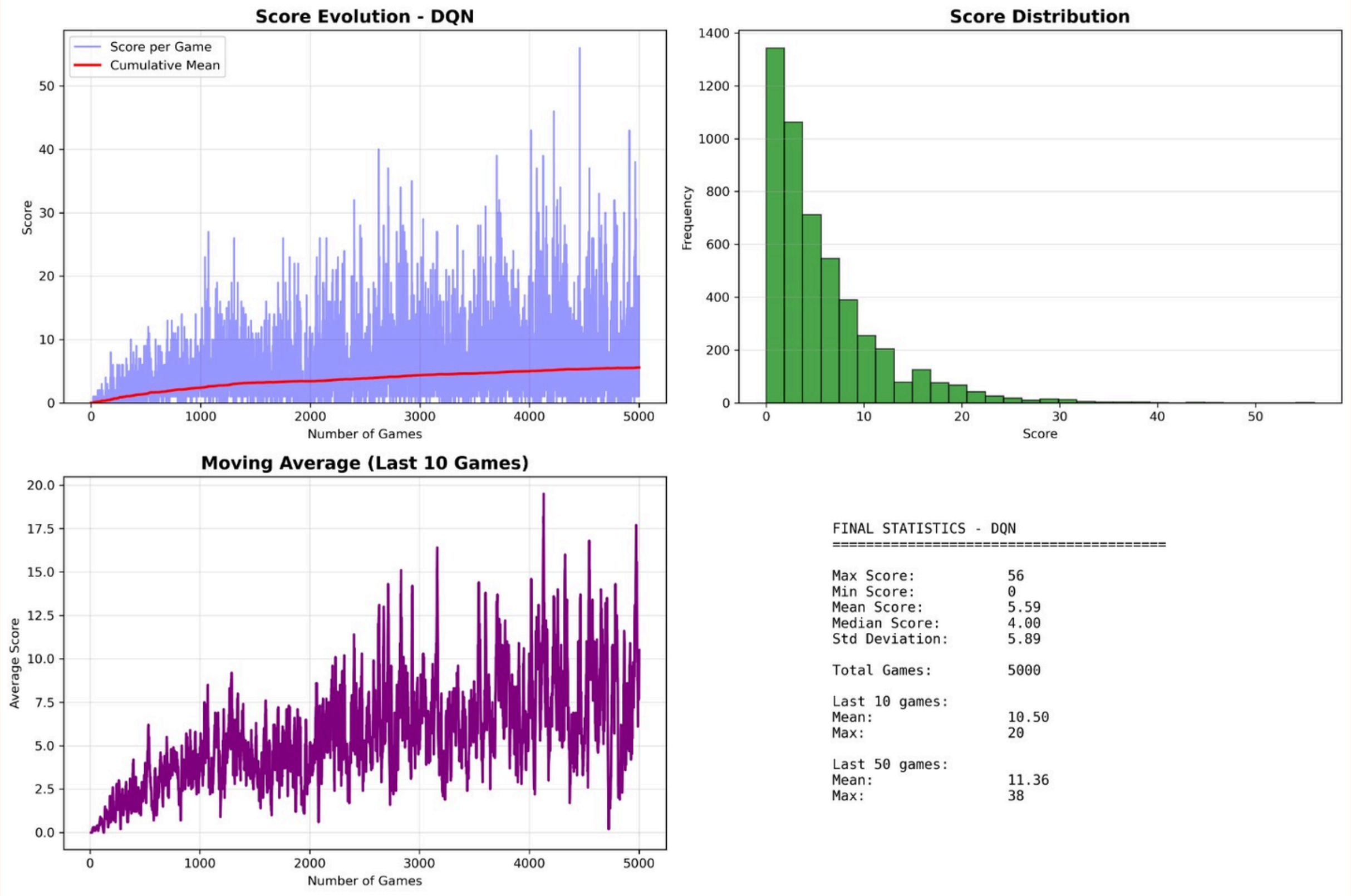
DEEP Q-LEARNING ESTE UN ALGORITM CARE COMBINĂ Q-
LEARNING-UL CLASIC CU REȚELE NEURONALE
DEEP LEARNING.

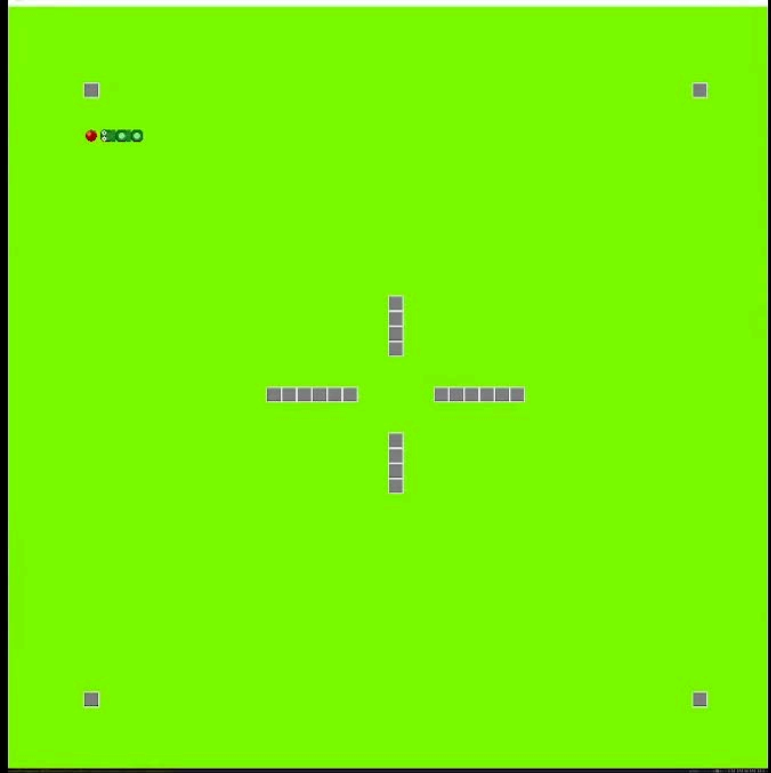
ÎNVAȚĂ O FUNCȚIE Q(STATE, ACTION) CARE ESTIMEAZĂ "CÂT
DE BUNĂ" ESTE O ACȚIUNE ÎNTR-O ANUMITĂ STARE.

Hiperparametru	Valoare	Rol
Gamma (γ)	0.9	Factor de discount (Balanță prezent vs. viitor)
Epsilon Start (ϵ)	1.0 (100%)	Explorare totală la începutul antrenamentului
Retea Neurală	256 neuroni	Capacitatea de a învăța pattern-uri complexe
Batch Size	1000	Numărul de experiențe folosite la fiecare pas de învățare
Replay Memory	100,000	Buffer pentru stocarea și refolosirea experiențelor
Learning Rate (α)	0.001	Pasul de ajustare a ponderilor rețelei (Viteza de învățare)



AGENT 2: DQN

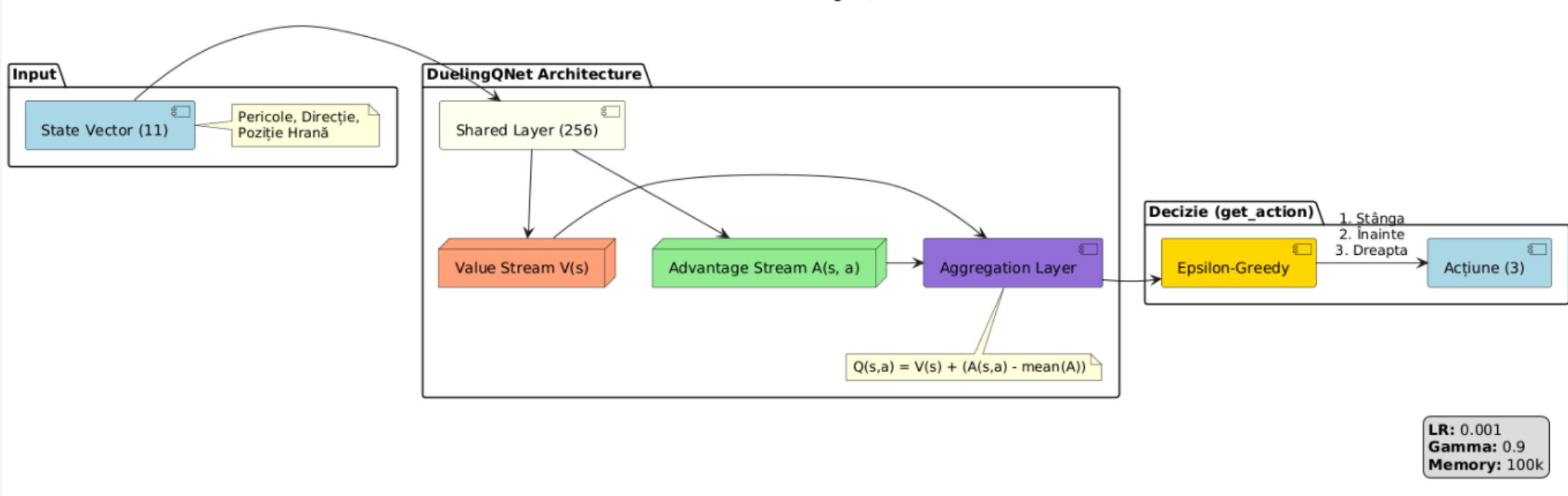




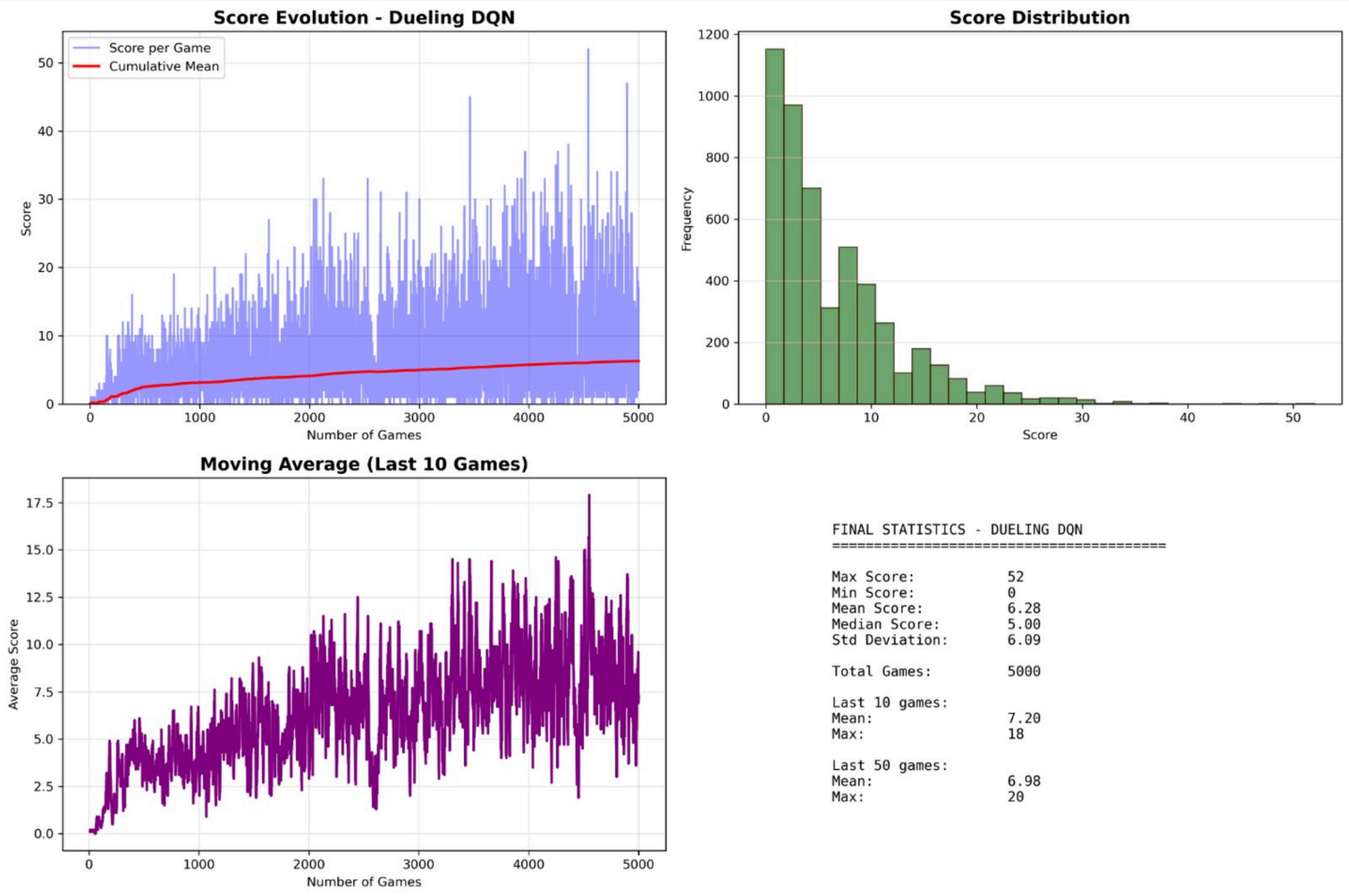
AGENT 3: DUELING DQN

Dueling Q-Learning (Dueling DQN) este o variantă de Q-Learning cu rețele neurale care estimează mai eficient valorile separând două componente: valoarea stării ($V(s)$) și avantajul acțiunii ($A(s,a)$). În loc să învețe direct ($Q(s,a)$), modelul are două „ramuri”: una pentru „cât de bună e starea” și una pentru „cât de bună e acțiunea față de celelalte”. Apoi le combină pentru a obține ($Q(s,a)$). Ajută mai ales când multe acțiuni sunt similare și diferențele dintre ele sunt mici.

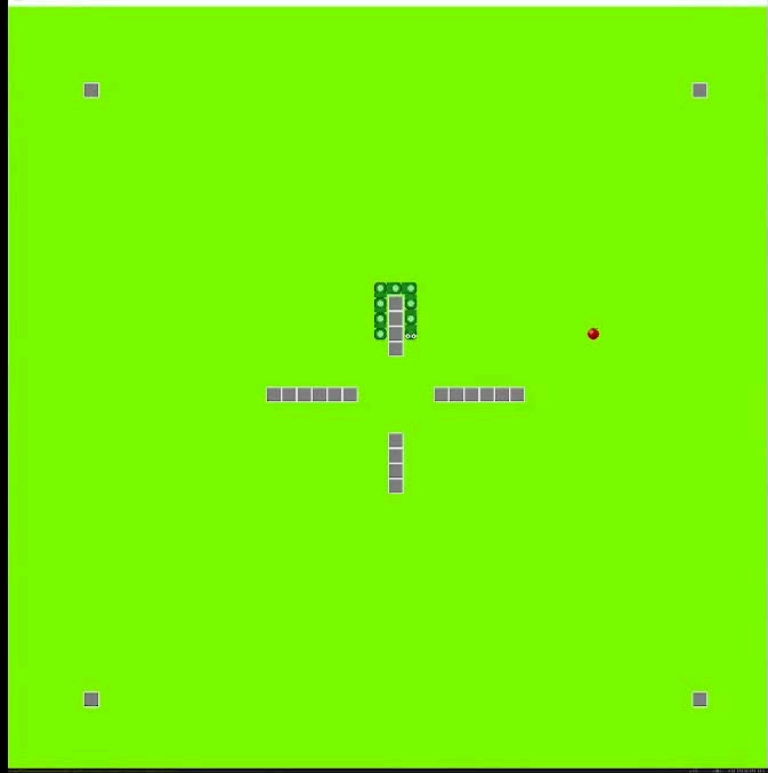
Learning Rate (α)	0.001	Viteza de actualizare a ponderilor rețelei
Epsilon Start (ϵ)	80	Nivelul inițial de explorare (scade per joc)
Epsilon Min	0	Limita minimă de explorare
Replay Memory	100,000	Capacitatea de stocare a experiențelor (deque)
Batch Size	1,000	Experiențe antrenate la fiecare pas
Hidden Layers	256	Dimensiunea stratului ascuns în DuelingQNet



AGENT 3: DUELING DQN



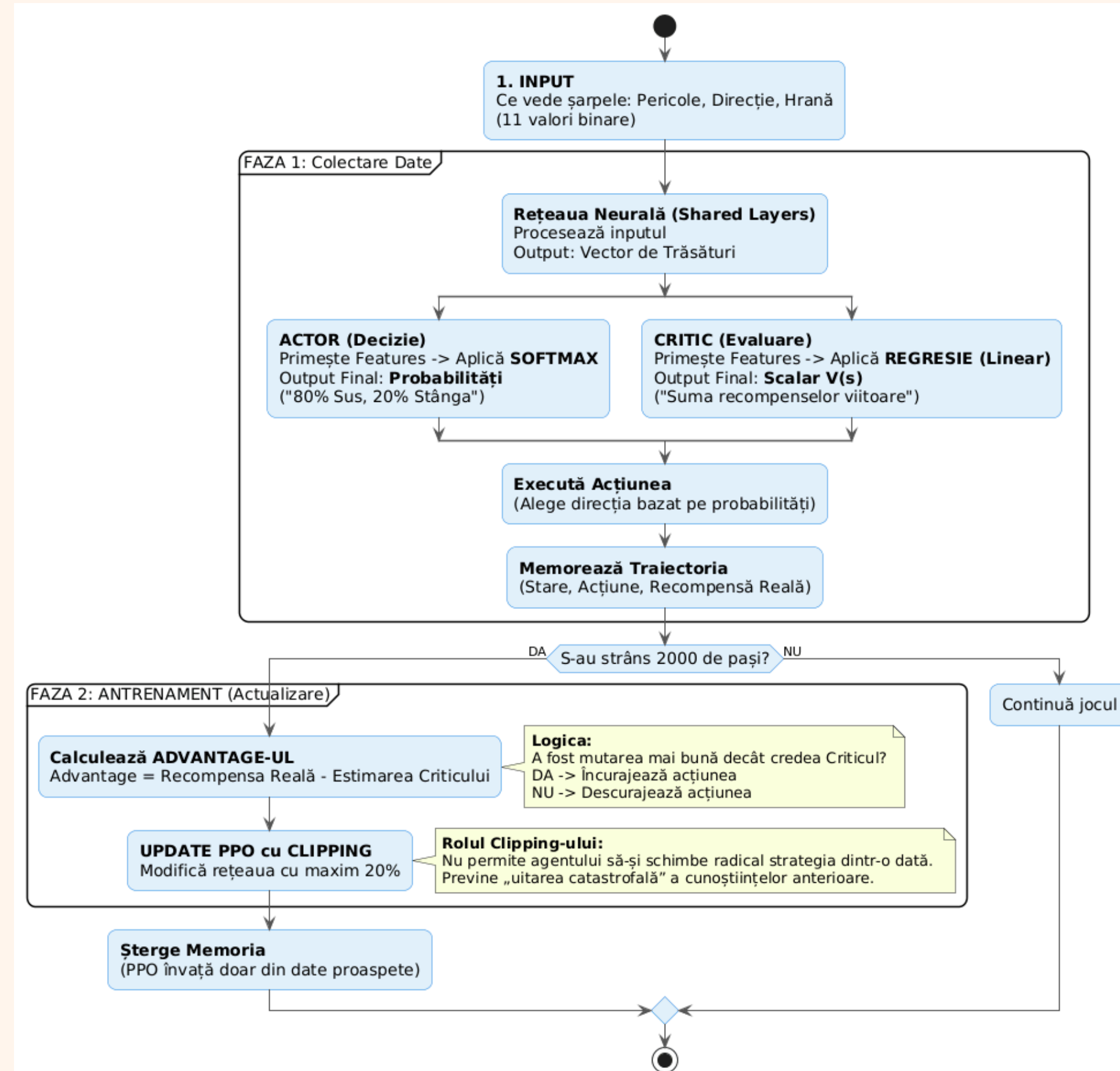
AGENT 4: PPO



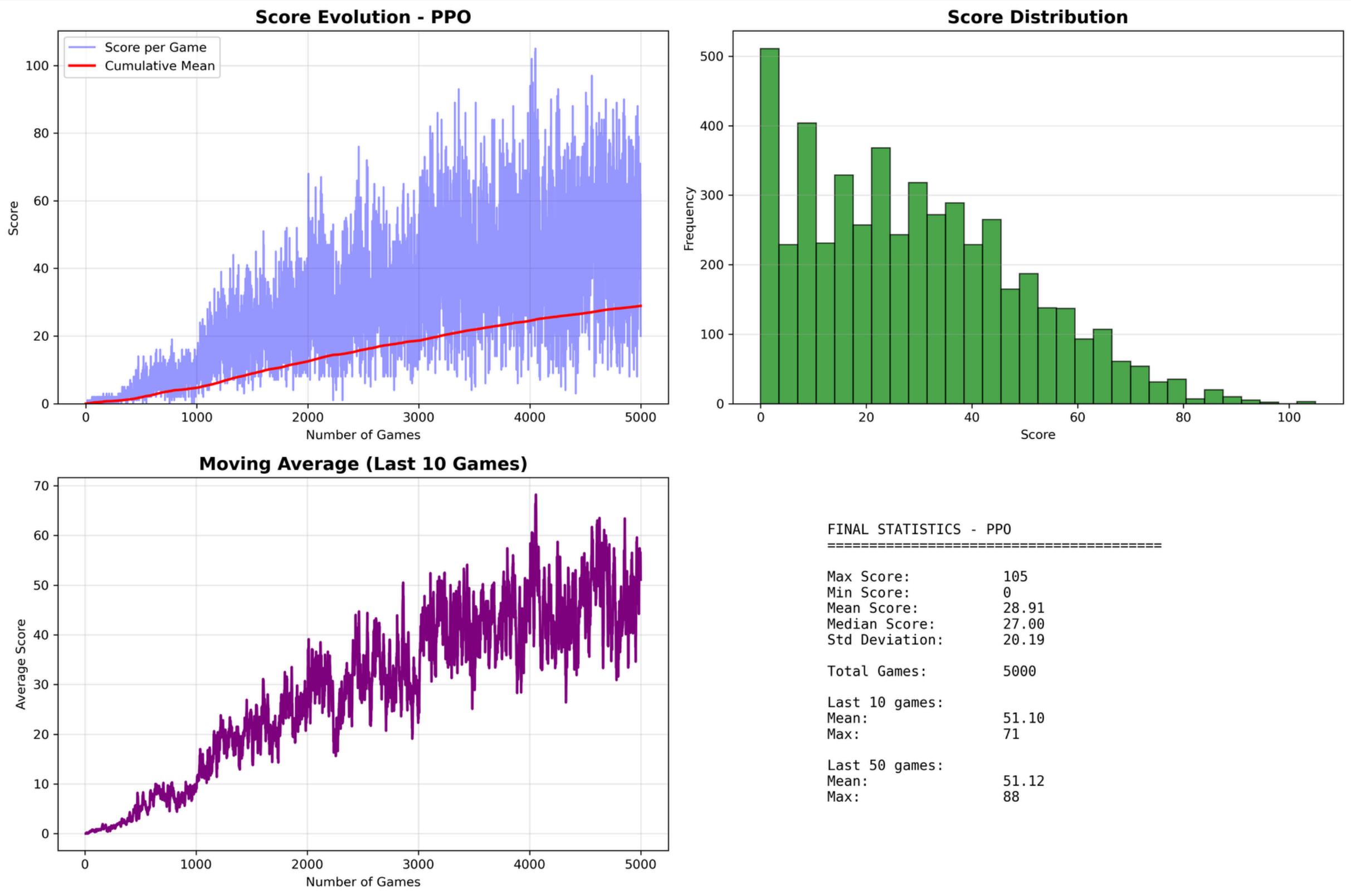
PPO este un agent hibrid de tip Actor-Critic care, spre deosebire de algoritmi anteriori, optimizează direct Politica de joc, nu doar estimarea recompenselor.

Folosește un mecanism de Clipping pentru a garanta o evoluție stabilă, eliminând riscul de a-și distruge strategia deja învățată printr-o schimbare bruscă.

Hiperparametru	Valoare	Rol
Gamma (γ)	0.99	<i>Focus Termen Lung</i> (Evită lăcomia imediată)
Clip Range (ϵ)	0.2	<i>Stabilitate</i> (Previne schimbările haotice)
Entropy Coeff	0.01	<i>Explorare</i> (Previne blocarea în bucle)
Learning Rate (α)	3×10^{-4}	<i>Pasul de Învățare</i> (Cât de fin se ajustează rețeaua)
Update Horizon	2000	<i>Memorie Batch</i> (Analiză traiectorie)
Hidden Layers	2 straturi $\times$ 256	<i>Inteligență</i> (Puterea de a procesa combinații complexe)



AGENT 4: PPO



REZULTATE

Double Q-Learning a eșuat în generalizarea pe hărți mari.

Fiind o metodă bazată pe memorare exactă (Lookup Table), agentul nu a putut transfera experiența acumulată dintr-o zonă a hărții în alta (nouă), tratând fiecare coordonată ca pe o problemă complet nouă.

DQN și **Dueling DQN** au demonstrat capacitatea de generalizare, dar lipsa consistenței.

Fiind metode bazate pe aproximarea valorii, agenții nu au putut garanta consistența performanței, transformând erorile minore de predicție în fluctuații majore de scor.

PPO a demonstrat nu doar capacitatea de generalizare, dar și consistența care lipsea.

Fiind o metodă bazată pe optimizarea directă a politicii, agentul a reușit să elimine fluctuațiile cauzate de erorile de estimare, consolidând un comportament care a garantat o performanță constantă, nu doar ocazională.

REZULTATE

